Seed-Based Deanonymization Via Databricks & Spark — Project Report

Rebecca Alvarez
CSC 4311 - Cloud Computing

1. Motivation

Social network graphs are routinely published for research after stripping user-names and replacing them with random pseudonyms. Narayanan and Shmatikov (2009)[3] demonstrated that this pseudonym-based anonymization is insufficient. Given a small set of known seed mappings between the original and anonymized graphs, an attacker can iteratively propagate those seeds through the shared neighborhood structure and recover the identity of most remaining nodes.

Last year in Privacy Course, CSC 4223, I was asked to create a program that uses their algorithm as a base in order to deanonymize a given graph. When tested on a new dataset for this project, it reported an acceptable accuracy rate, but relies on Python's multiprocessing library and NetworkX adjacency lookups, which scale poorly. On a 7,126-node Twitch friendship graph (35,324 edges), the algorithm takes over 3 hours with 20 CPU workers and nearly 24 hours with 4. On a 168,114-node graph with 6.8 million edges, it is completely infeasible on a single machine.

For this project, I aim to utilize cloud-distributing computing, namely Apache Spark on AWS Databricks, to improve the runtime of the program, then leverage the decreased runtime to improve the overall accuracy. The final goal is to successfully deanonymize the 6.8 million edge graph. Five incremental experiments were done to work towards this final goal, and the resulting system successfully deanonymized the large graph in under 16 minutes with 93.5% accuracy, compared to the original implementation's 3+ hours on a graph significantly smaller.

2. System Design

2.1. Problem Statement

Given: The original graph and the anonymous graph as edge lists, G1 & G2

A partial seed mapping of known node connections, $S \subset V(G1)*V(G2)$

A ground-truth mapping for validation, T

The goal is to expand S by identifying additional pairs using the known connections and neighbors of the mapped nodes within the initial set of S. This is a form of seed-based percolation graph matching[4]. Known seeds "percolate" out through shared neighborhood structure, which in turn identify more correspondences.

2.2. Algorithm Structure

All variants of the implementations share the same steps:

1. For each unmatched G1 node u, find its neighbors that are already in S
2. For each unmatched G2 node v, find its neighbors that are already in S
3. For each candidate pair (u, v), compute a similarity score based on their shared mapped neighbors
4. Select the best matches (one-to-one) above a confidence threshold
5. Add new matches to S and repeat until convergence

The experiments improve upon this base framework in three methods:

Method	Evolution of Changes
Change infrasturcutre	Local Python → Databricks Serverless → Classic single-node → Classic multi-node
Change similarity measure	Eccentricity/cosine → Jaccard index → Adamic-Adar index
Optimize for larger graphs	None → bulk joins → broadcast seeds, hub cap, repartitioning, auto-threshold

The following design decisions apply across all Spark variants:

1. **One-to-one matching via mutual best-match.** Introduced in Cell 8 (Jaccard) and carried forward. Uses `left_anti` joins on both sides to prevent collisions.
2. **Undirected edges.** Both graphs are made undirected via `union(select(v,u))` so that neighbor lookups work regardless of edge direction in the source file.

2.3. Datasets

Both graphs are sourced from Stanford Network Analysis Project, specifically MUSAE's Twitch Dataset for the English community[9], and the Twitch Gamers dataset pulled from the public API in 2018[8]. They will be referred to as the small and large graph, respectively.

Graph	Source	Nodes	Edges	Avg. Degree	Given Seeds	Seed %
Small	MUSAE	7,126	35,324	9.91	500	~7%
Large	Twitch Gamers	168,114	6,797,557	80.9	12,000	~7%

Graphs are anonymized via pseudonym permutation. The anonymization program returns the complete ground-truth mapping, and partial seed mappings were generated by creating a copy and deleting entries.

2.4. Compute Environments

Environment	Specs	Usage
Local #1	8th Gen Intel i7-8565U, 4 logical cores, 16 GB DDR4	Control group (Cell 3)
Local #2	12th Gen Intel i7-12700K, 20 logical cores, 16 GB DDR4	Control group comparison
Databricks Serverless	Intel Xeon 8375C, 4 cores, 15.3 GB, Spark Connect	Cell 5 (failed)
Classic Single-Node	i3.xlarge: 4 vCPUs, 30.5 GB RAM, 950 GB NVMe SSD	Cells 6, 8, 9

Classic Multi-Node	r5d.xlarge — 2–8 workers, Photon, identical specs as above across each worker	Cells 10, 11

3. Implementation

Implementation is contained within a single Databricks notebook, `CloudComputingProject Notebook.ipynb`, with 11 algorithm cells, each representing an iteration of the system leading up to the final version. Shared utility functions exist in Cell 7 which are used by all the cells following it. The results of each cell will be combined in a single table in Section 4.

3.1. Control Group - Local Python Multiprocessing (Cell 3)

The baseline implementation uses NetworkX for graph representation and Python's multiprocessing.Pool for parallelism. Each worker processes one G1 node at a time:

- **Graph library**: NetworkX adjacency dicts
- **Parallelism**: multiprocessing.Pool with Manager and Lock for shared state
- **Scoring**: For each G1 node, iterate over all G2 nodes, compute $Score(u, v^i) = \frac{sharedMappedNeighbors}{(\sqrt{degree\ of\ u}*\sqrt{degree\ of\ v})}$ and rank by $Eccen = \frac{(max-max2)}{std}$
- **Seed updates**: `sharedSeedMappings.extend()` with a process lock

This has O(n²) complexity per iteration (every G1 node checks every G2 node). It also does not run on Databricks as `multiprocessing.Pool, Manager,` and `Lock` are incompatible with both Serverless and Spark environments. It is included in the notebook as a read-only reference.

3.2. Test #1 - Change Infrastructure By Porting to Spark (Cells 4-6)

Cell 4 (Serverless, multiprocessing attempt): The first cloud attempt was to run the original Python multiprocessing code directly on Databricks Serverless. This failed immediately. As previously mentioned, `multiprocessing.Pool, Manager,` and `Lock` are incompatible with Serverless compute and Spark Connect environments, which do not expose shared memory across workers. This incompatibility motivated the full rewrite from Python parallelism to native PySpark DataFrames.

Cell 5 (Serverless): A direct port replacing NetworkX with PySpark DataFrames. The system OOMs after one iteration on 15.3 GB Serverless. The key problem being how `score_numerator()` and `score_denominator()` each call `.filter().count()` *per node pair*, triggering a separate Spark job for every (G1, G2) candidate. For 7,126*7,126 = 50M+ pairs, this generates billions of Spark jobs.

Cell 6 (Classic single-node): Moves away from Serverless and also replaces per-node loops with bulk Spark joins. Instead of iterating over individual node pairs:

1. Join G1 edges with seeds to get all `(leftNode,mappedRightNeighbor)` pairs at once
2. Join G2 edges with seeds to get all `(rightNode,mappedRightNeighbor)` pairs
3. Join these two on `mappedRightNeighbor` to find all shared mapped neighbors in a single pass

4. `groupBy("leftNode", "rightNode").agg(count(*))` to compute scores for all candidate pairs simultaneously

Additional changes:

- **Undirected edges:** `g1_edges.union(g1_edges.select(col("v").alias("u"), col("u").alias("v")))` ensures both directions are represented
- **Cached degrees:** Computed once via `groupBy("u").count()` and reused across iterations
- **Lineage breaking:** `localCheckpoint(eager=True)` after each iteration prevents the Spark DAG from growing unboundedly
- **While loop:** Replaces recursive `deanonymize_spark()` calls which deters risk of stack overflow
- **Removed iterative edge filtering:** Early experiments filtered matched nodes' edges as an attempt to decrease the size of the graph each iteration, but this broke propagation paths and dropped accuracy from 60% to 25%. Switching to bulk Spark joins served as a better optimization strategy, and keeping the graph intact raised accuracy to 55–65%.

3.3. Test #2 - Change Similarity Measure (Cells 8, 9)

Before changing the similarity measure, a sub-experiment raised the eccentricity confidence threshold from 0.5 to 1.0. This alone improved accuracy from 55.42% to 64.82% (+13%), increased matches by 670 (3,949 → 4,619), and also reduced runtime by 25.6% (429s → 319s) because fewer false matches meant fewer seeds to propagate. However, the eccentricity method of measuring similarity still had room for improvement in both aspects, accuracy and runtime. Since runtime was improved substantially by the change in infrastructure, tests to compare similarity measures could be performed.

Cell 8 (Jaccard Similarity): Replaces the eccentricity confidence metric with the Jaccard index:

$$J(u,v) = \frac{Intersection(u,v)}{Union(u,v)} = \frac{|sharedMappedNeighbors|}{(|mappedCount\ of\ u|+|mappedCount\ of\ v|-|shared|)}$$

Also introduces mutual best-match selection to prevent collisions:

1. Rank by best rightNode per leftNode `(Window.partitionBy("leftNode").orderBy(desc("jaccard")))`
2. Among those winners, rank by best leftNode per rightNode
3. Only accept pairs that are each other's top choice

The main limitation behind the Jaccard index is how it only views the mappings from a purely quantitative aspect. All relationships between nodes, regardless if that node has more or less connections, are weighted the same and it does not differentiate.

Cell 9 (Adamic-Adar): Replaces Jaccard with the Adamic-Adar index[1]:

$$AA(u,\ v)\ =\ \Sigma_{w\ \in\ sharedMappedNeighbrs}\ \frac{1}{ln(deg(w))}$$

Jaccard treats all shared neighbors equally, but a shared neighbor of degree 3 is far more informative than a hub of degree 500[2]. The AA index accounts for this and weights each shared neighbor inversely by the log of its degree, so low-degree (specific) connections

contribute more to the score. Neighbors with degree ≤ 1 are excluded to prevent division by 0.

3.4. Test #3 - Single-Node → Multi-Node (Cell 10)

Changing to a multi-node cluster required replacing `localCheckpoint` with a parquet write/read on a Unity Catalog Volume to serve as the lineage-breaking mechanism:

```
seeds_df.write.mode("overwrite").parquet(SEEDS_TEMP_PATH)

seeds_df = spark.read.parquet(SEEDS_TEMP_PATH)
```

On the small graph, this achieved 88.86% accuracy in ~3 minutes. On the large graph, iteration 1 took 61 minutes and iteration 2 crashed as there were no optimizations in place for a graph of that size. This exposed several scale bottlenecks: no hub cap (join explosion from high-degree nodes on iter 2), no seed broadcast (full shuffle every iteration), and Python `open()` for file loading (driver OOM on 6M-edge files). The latter was still included mainly as an artifact from previous iterations of the system.

3.5. Tests #4 & #5 - Large-Graph Optimizations (Cell 11)

Cell 11 is composed of two tests. Test #4 focused on using the decreased runtime to tune the accuracy by adjusting the threshold value of the AA index. Thresholds values were first tested empirically starting from 0.5 and testing surrounding values 0.1, 0.3, 0.7, and 1.0. Instead of continuing to randomly pick threshold values, a formula for the threshold was also derived from the AA weight structure, and Youden's J statistic[7] was looked into as a threshold choice and comparison to the derived formula.

Test #5 applied several other optimizations to the large graph in addition to the threshold formula. The optimizations were tested on the small graph first to ensure function.

1. **Spark-native file loading:** Replaced `open()→spark.createDataFrame()` with `spark.read.text()` plus column splitting, eliminating the driver memory bottleneck:

   ```
   spark.read.text(file_path).select(

   spark_split(trim(col("value")),
   "\\s+")[0].cast("int").alias("u"),

   spark_split(trim(col("value")),
   "\\s+")[1].cast("int").alias("v")

   )
   ```

2. **Broadcast seeds.** `broadcast(seeds_df)` gives every worker a local copy of the seed mapping, eliminating the shuffle that previously moved seed data across the network on every join.
3. **Early exclusion.** Already-matched nodes are filtered from `g1_mapped` and `g2_mapped` *before* the shared-neighbor join, not after. This reduces the join input size substantially.
4. **Hub degree cap (800).** Nodes with degree > 800 are excluded from the shared-neighbor computation. A node with 5,000 connections contributes 1/ln(5000) ≈ 0.117 AA weight but explodes the join because it appears in thousands of candidate pairs. 1,502 hub nodes were skipped in the large graph.

5. **Dynamic shuffle partitions (800).** Tuned for the ~12M undirected edges: 800 partitions yields ~15K edges per partition, balancing worker utilization against partition overhead. Heuristic: ~1 partition per 7,500 undirected edges.
6. **Edge repartitioning.** `repartition(n_parts, "u")` co-locates all edges from the same source node on the same worker, eliminating subsequent shuffles during mapped-neighbor joins.
7. **Minimum shared neighbors (2).** Candidate pairs with fewer than 2 shared mapped neighbors are filtered before the expensive AA weight computation. This is a cheap `groupBy.count()` pre-filter that eliminates noise. For sparse graphs, ≥1 is recommended instead.
8. **Coalesced writes.** `coalesce(max(1, n_parts // 10))` reduces the number of parquet files written during lineage breaking, lowering filesystem overhead.

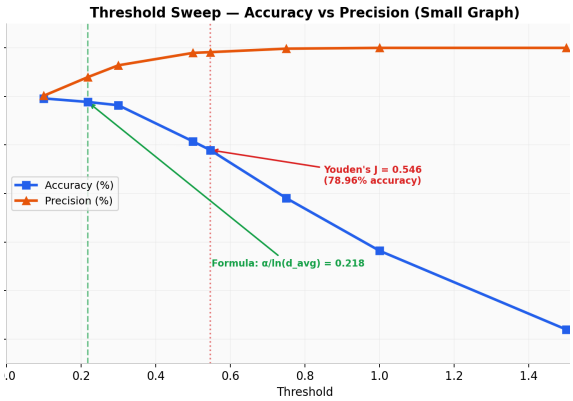
3.6. Auto-Threshold Formula

Instead of using a fixed score threshold, Cell 11 derives the threshold from the graph's average degree. Derivation of this formula is composed of two steps:

1. **Expected weight per neighbor:** From the AA formula, each sharedMappedNeighbor w contributes $\frac{1}{ln(deg(w))}$. Jensen's Inequality[6] can be applied since $\frac{1}{ln(x)}$ is convex for x>1: $E[\frac{1}{ln(deg)}] \geq \frac{1}{ln(E[deg])} = \frac{1}{ln(avgDeg)}$
2. **Set threshold as α neighbors of evidence:** A pair with k sharedMappedNeighbors scores approximately $\frac{k}{ln(avgDeg)}$. Requiring α neighbors of evidence creates a formula for threshold as: $threshold = \frac{\alpha}{ln(avgDeg)}$.

   $\alpha = 0.5$ means pairs must have at least half a neighbor's worth of evidence, which is permissive since the mutual best-match constraint that has been carried over already provides precision.

The threshold sweep shows a clear connection between the choice of threshold. Lower threshold increases accuracy, but decreases precision and increases runtime. The formula threshold of 0.218 maximizes the gain in accuracy without losing too much precision. Testing below it at 0.1 gained only +0.73% accuracy but lost 3.83% precision and took 4.4x longer (50 iterations vs 13).



Auto-scaling verification:

Graph	avgDeg	threshold	Per-neighbor weight	Resulting Accuracy
Small (7k nodes)	9.91	0.5/ln(9.91) = 0.218	~0.44	88.86%
Large (168k nodes)	80.9	0.5/ln(80.9) = 0.114	~0.23	93.50%

Comparison to Youden's J: Youden's J, which finds the threshold by maximizing the vertical distance between the ROC curve and the random-guess diagonal, comes up with a threshold of 0.546 on the small graph. This produces slightly higher precision, but sacrifices a lot of accuracy.

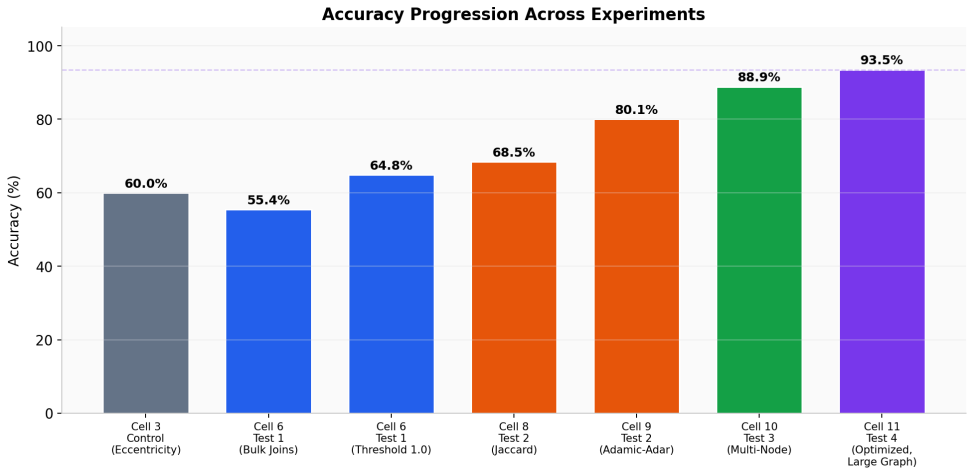
Method	**Threshold**	**Accuracy**	**Precision**	**Runtime**
Youden's J	0.546	78.96%	99.12%	96s
Formula ($\alpha = 0.5$)	0.218	88.86%	93.97%	178s

Youden's J underperforms in this context because it optimizes for a single-shot classifier and doesn't account for the iterative propagation nature of this context. The conservative threshold starves the propagation loop and can leave out potential matches. The lower threshold of the formula feeds the propagation while counting on the mutual best-match to keep precision intact.

4. Evaluation

4.1. Accuracy Progression

Cell	**Test**	**Algorithm**	**Graph**	**Accuracy**	**Matches**
3	Control	Eccentricity (networkx)	Small	60.00%	6,140/7,126
5	Test 1	Eccentricity (PySpark)	Small	N/A	OOM after iter 1
6	Test 1	Eccentricity (with edge filtering)	Small	25.13%	1,791/7,126
6	Test 1	Eccentricity (bulk joins, threshold=0.5)	Small	55.42%	3,949/7,126
6	Test 1	Eccentricity (bulk joins, threshold=1.0)	Small	64.82%	4,619/7,126
8	Test 2	Jaccard Similarity	Small	68.45%	4,878/7,126
9	Test 2	Adamic-Adar (single- node)	Small	80.07%	5,705/7,126
10	Test 3	AA (multi-node)	Large	N/A	Crashed iter 2
11	Test 4	AA + optimizations	Small	88.86%	6,332 / 7,126
11	Test 5	AA + optimizations	Large	**93.50%**	157,179/168,114



The accuracy improvement from 60%→93.5% has two main factors:

1. **Similarity measure:** Eccentricity → Jaccard → Adamic-Adar. Each change in measure improved the match quality by weighting shared neighbors more meaningfully. The mutual best-match from Cell 8 also eliminated collision errors that could have reduced accuracy as well.
2. **Graph density:** The large graph's higher density's higher degree of 80.9 vs 9.91 provides more shared-neighbor evidence per candidate pair, which generates more higher-quality matches[5]. This aligns with percolation theory predictions[4] that state denser graphs propagate seeds faster and more accurately.

4.2. Runtime Progression

Cell	Test	Cluster	Graph	Runtime
3	Control	Local (4 workers)	Small	23h 45m
3	Control	Local (20 workers)	Small	3h 17m
5	Test 1	Serverless	Small	OOM (iter 1)
6	Test 1	i3.xlarge single-node	Small	~5 min
8	Test 2	i3.xlarge single-node	Small	376s
10	Test 3	r5d.xlarge multi-node	Large	Crashed iter 2 (iter 1=61 min)
11	Test 4	r5d.xlarge multi-node	Small	~178s (3 min)
11	Test 5	r5d.xlarge multi-node	Large	**918s** **(15.3 min)**

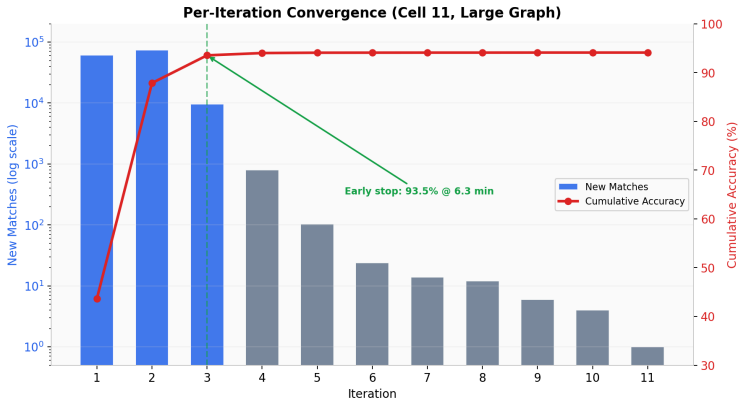
Baseline vs. After Optimization (Large Graph):

Run	Threshold	Iter 1 Time	Full Runtime	Completed?
Baseline #1 (Cell 10)	0.5	3,678s (61 min)	Never finished	No
Baseline #2 (Cell 10)	1.5	4,069s (68 min)	Crashed iter 2	No
Optimized (Cell 11)	0.114 (auto)	187s (3.1 min)	918s (15.3 min)	Yes

Per-iteration Breakdown from the final system (Cell 11):

Iteration	New Matches	Cumulative Seeds	Progress	Iter Time	Cumulative Time
1	61,342	73,360	43.64%	197.6s	198.1s
2	74,281	147,676	87.84%	122.0s	320.8s
3	9,531	157,224	93.52%	59.8s	381.0s
4	793	158,017	93.99%	71.3s	452.8s
5	103	158,120	94.06%	60.9s	514.1s
6	24	158,144	94.07%	55.5s	570.9s
7	14	158,158	94.08%	47.0s	618.2s
8	12	158,170	94.08%	48.5s	667.2s
9	6	158,174	94.09%	46.5s	714.8s
10	4	158,178	94.09%	58.3s	773.5s
11	1	158,179	94.09%	46.9s	820.7s
12	0(converged)	-	-	-	918.2s

The first two iterations account for 87.84% of the coverage and 93.5% accuracy is reached by the third iteration, 6.3 minutes into the runtime. The remaining iterations only contribute +0.57% coverage, so early stopping after the third iteration is an acceptable strategy for more time-sensitive applications, such as limiting cloud resource usage time.



4.3. Final System Results

Metric	Value
Accuracy	93.50%
Precision	99.37%
Calculated seeds	158,179
Correct matches	157,179
False matches	1,000
Coverage	94.09% (158,179 / 168,114 nodes matched)
Unmatched nodes	9,935
Runtime	918.2s (15.3 min)
Iterations	12 (last new match at iteration 11)
Threshold	0.1138 (auto-computed: 0.5 / ln(80.9))
Cluster	r5d.xlarge, multi-node

4.4. Small vs Large Graph: Final Results

Metric	Small (7k nodes)	Large (168k nodes)
Avg Degree	9.91	80.9
Threshold	0.218	0.114
Accuracy	88.86%	93.50%
Precision	93.97%	99.37%
Runtime	178s (3 min)	918s (15.3 min)

5. Conclusion

This project demonstrates the strength of cloud-based distributed computing in the application of graph processing. It can change graph deanonymization from a multi-hour local computation, to a scalable system with more practical runtimes. Starting from Narayanan and Shmatikov's original seed-based algorithm, which required 3 hours and 17 minutes on a 7,126-node graph using 20 local CPU workers, the system was iteratively improved across five main tests that resulted in a large-graph optimized Spark implementation that deanonymizaes a 168,114-node graph with almost 7 million edges in just 15.3 minutes while boasting a 93.5% accuracy and 99.4% precision.

Three key contributions resulted in this change. First, the change from nested Python loops to bulk Spark joins resulted in a 40x speedup on the small graph and also made the large graph feasible to process for the first time. Second, replacing the original eccentricity-based similarity scoring metric with the Adamic-Adar similarity index

allowed shared neighbors to be weighted properly according to their degree with smaller degree neighbors being weighted more highly, which increased accuracy from 60% to 88% on the small graph. Lastly, the auto-threshold formula $\frac{\alpha}{ln(avgDeg)}$ produces a system that eliminates the need for emphirical AA threshold testing on new datasets, and returns high accuracy while not sacrificing precision.

Eight targeted optimizations, broadcast seeds, hub degree cap, Spark-native file loading, early exclusion, dynamic partitioning, edge repartitioning, minimum shared neighbor filtering, and coalesced writes, collectively achieved a further ~20x speedup on the large graph's first iteration and allowed it to fully run to completion.

Notably, the 168K-node graph achieved higher accuracy than the 7K-node graph (93.5% vs 88.9%), confirming the percolation theory prediction that denser graphs provide more structural evidence for matching[4]. Early-stopping analysis showed 93.5% accuracy is reached by iteration 3 in 6.3 minutes, with the remaining 9 minutes adding less than 0.6 percentage point, making early termination viable for time-sensitive contexts.

This system still has its limits. 9,935 unmatched nodes (5.9%) and 1,000 false matches stem from node that are unreachable by percolation and the threshold's trade of less false positives for higher recall. Future work could further improve the system by using methods such as multi-hop neighborhood features or neural networks. The confidence of the auto-threshold could also be expanded upon with further testing on different datasets.

6. References

[1] L. A. Adamic and E. Adar, "Friends and neighbors on the Web," Social Networks, vol. 25, no. 3, pp. 211–230, 2003. - Original AA similarity definition.

[2] D. Liben-Nowell and J. Kleinberg, "The link-prediction problem for social networks," JASIST, vol. 58, no. 7, pp. 1019–1031, 2007. - AA vs other metrics evaluation.

[3] A. Narayanan and V. Shmatikov, "De-anonymizing social networks," in Proc. IEEE S&P, 2009, pp. 173–187. — Foundational seed-based graph deanonymization.

[4] L. Yartseva and M. Grossglauser, "On the performance of percolation graph matching," in Proc. ACM COSN, 2013, pp. 119–130. - Percolation theory; denser graphs propagate faster.

[5] P. Pedarsani and M. Grossglauser, "On the privacy of anonymized networks," in Proc. ACM SIGKDD, 2011, pp. 1235–1243. - Degree distribution affects matching confidence.

[6] G. R. Grimmett and D. R. Stirzaker, *Probability and Random Processes*, 3rd ed. Oxford University Press, 2001. - Jensen's inequality.

[7] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, vol. 3, no. 1, pp. 32–35, 1950. - Threshold selection via sensitivity/specificity optimization.

[8] B. Rozemberczki and R. Sarkar, "Twitch Gamers: a Dataset for Evaluating Proximity Preserving and Structural Role-based Node Embeddings," *arXiv preprint arXiv:2101.03091*, 2021. - Dataset for testing node embedding performance on social graphs - "Large" Graph Dataset

[9] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-scale Attributed Node Embedding," *arXiv preprint arXiv:1909.13021*, 2019. - "Small" Graph dataset